Octo: An Open-Source Generalist Robot Policy

Octo Model Team

Dibya Ghosh*,[1] Homer Walke*,[1] Karl Pertsch*,[1,2] Kevin Black*,[1] Oier Mees*,[1]

Sudeep Dasari[3] Joey Hejna[2] Tobias Kreiman[1] Ria Doshi[1] Charles Xu[1] Jianlan Luo[1] You Liang Tan[1]
Lawrence Yunliang Chen[1] Pannag Sanketi[4] Quan Vuong[4] Ted Xiao[4] Dorsa Sadigh[2] Chelsea Finn[2] Sergey Levine[1]

[1]UC Berkeley [2]Stanford [3]Carnegie Mellon University [4]Google Deepmind
https://octo-models.github.io

arXiv:2405.12213v2 [cs.RO] 26 May 2024

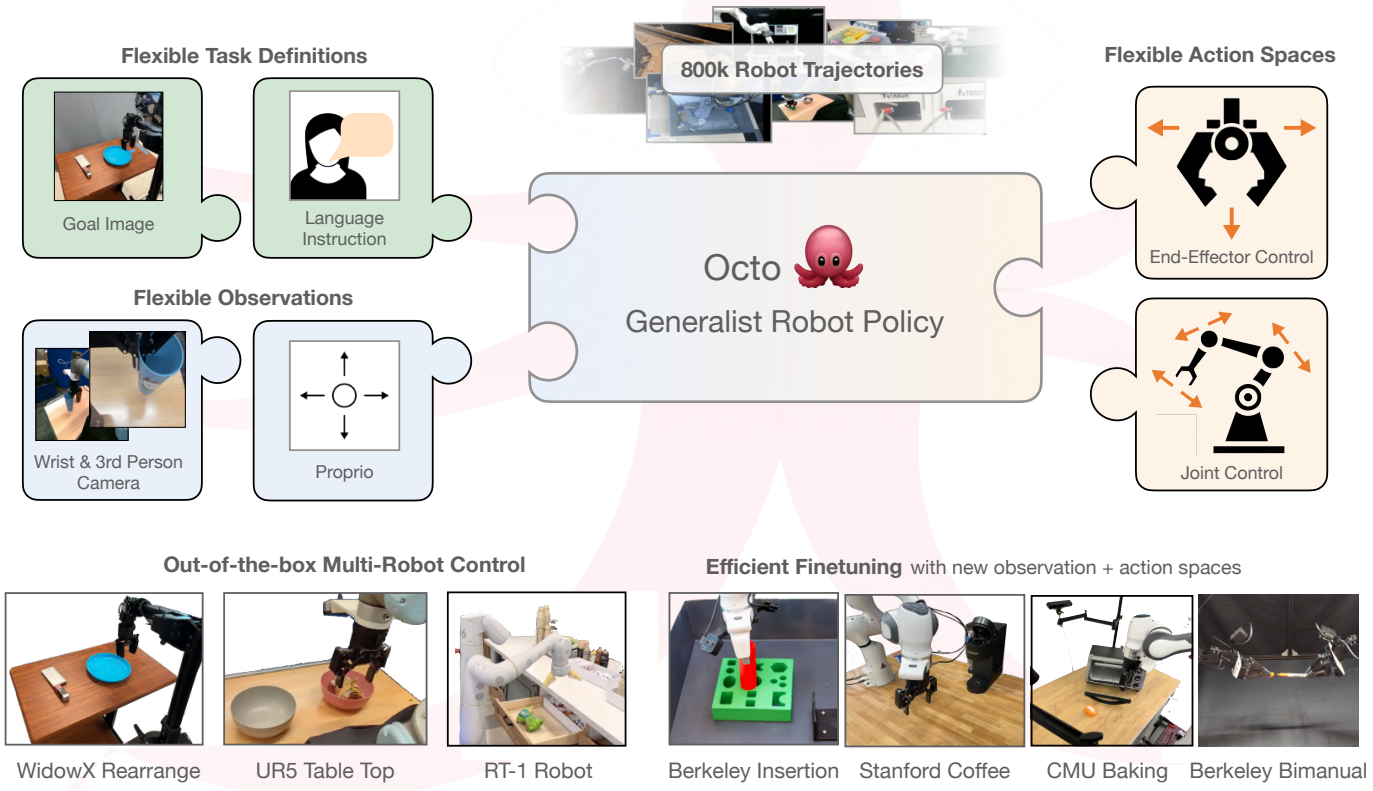

Fig. 1: We introduce Octo, an open-source, generalist policy for robotic manipulation. Octo is a transformer-based policy pretrained on 800k diverse robot episodes from the Open X-Embodiment dataset [67]. It supports flexible task and observation definitions and can be quickly finetuned to new observation and action spaces.

***Abstract*—Large policies pretrained on diverse robot datasets have the potential to transform robotic learning: instead of training new policies from scratch, such generalist robot policies may be finetuned with only a little in-domain data, yet generalize broadly. However, to be widely applicable across a range of robotic learning scenarios, environments, and tasks, such policies need to handle diverse sensors and action spaces, accommodate a variety of commonly used robotic platforms, and finetune readily and efficiently to new domains. In this work, we aim to lay the groundwork for developing open-source, widely applicable, generalist policies for robotic manipulation. As a first step, we introduce Octo, a large transformer-based policy trained on 800k trajectories from the Open X-Embodiment dataset, the largest robot manipulation dataset to date. It can be instructed via language commands or goal images and can be effectively finetuned to robot setups with new sensory inputs and action spaces within a few hours on standard consumer GPUs. In experiments across 9 robotic platforms, we demonstrate that Octo serves as a versatile policy initialization that can be effectively finetuned to new observation and action spaces. We also perform detailed ablations of design decisions for the Octo model, from architecture to training data, to guide future research on building generalist robot models.**

I. Introduction

The common approach for robotic learning is to train policies on datasets collected for the specific robot and task at hand. Learning from scratch in this way requires significant data collection effort for each task, and the resulting policies usually exhibit only narrow generalization. In principle, collected

*Lead authors, ordered alphabetically, see Section A for list of contributions. Correspondence to {dibya.ghosh, homer_walke, pertsch, kvablack, oier.mees}@berkeley.edu

experience from other robots and tasks offers a possible solution, exposing models to a diverse set of robotic control problems that may improve generalization and performance on downstream tasks. However, even as general-purpose models become ubiquitous in natural language [68, 88]) and computer vision [76, 44], it has proven challenging to build the analogous "general-purpose robot model" that can control many robots for many tasks. Training a unified control policy in robotics presents unique challenges, requiring handling different robot embodiments, sensor setups, action spaces, task specifications, environments, and compute budgets.

Towards this direction, several works have proposed robotic foundation models that directly map robot observations to actions and provide zero-shot or few-shot generalization to new domains and robots. We broadly refer to these models as "generalist robot policies" (GRPs), emphasizing their ability to perform low-level visuomotor control across tasks, environments, and robotic systems [75, 9, 23, 103, 10, 81, 1, 91, 35, 94, 45]. For example, the GNM model [80] generalizes across different robotic navigation scenarios, the RoboCat model [9] handles different robot embodiments for goal-conditioned tasks, and the RT-X model [67] performs language-conditioned manipulation across five robot embodiments. Although these models represent significant steps toward a true "general-purpose robot model," they have been limited in multiple important aspects: they typically constrain downstream users to a pre-defined and often restrictive set of input observations, e.g., a single camera stream; they lack support for effective finetuning to new domains; and importantly, the largest of these models are not available to the general public.

We design a system for pretraining generalist robot policies more suitable for the diversity of interfaces in downstream robotic applications. The core of our model is a transformer architecture that maps arbitrary input tokens (created from observations and tasks) to output tokens (then decoded into actions), which can be trained on a diverse dataset of robots and tasks. With no additional training, this policy can accept different camera configurations (e.g., workspace or wrist cameras), can control different robots, and can be guided via either language commands or goal images — all by simply changing which tokens are fed into the model. Most importantly, the model can be adapted to new robot setups with new sensory inputs, action spaces, or morphologies by adding appropriate adapters and finetuning with a small target domain dataset and an accessible compute budget.

Our primary contribution is Octo, a transformer-based policy pretrained on the largest robot manipulation dataset to date: 800k robot demonstrations from the Open X-Embodiment dataset [67]. Octo is the first GRP that can be effectively finetuned to new observations and action spaces and the first generalist robot manipulation policy that is fully open-source, including the training pipeline, model checkpoints, and data. Finally, while the individual components that comprise Octo — a transformer backbone, support for both language and goal image specification, and a diffusion head to model expressive action distributions — have been discussed in prior work, the particular combination of these components into a powerful generalist robot policy is unique and novel.

We demonstrate through extensive experiments on 9 robots across 4 institutions that our combined system leads to state-of-the-art performance for out-of-the-box multi-robot control for single and dual-arm manipulation tasks and that Octo can be used as an effective initialization for finetuning to unseen setups with new observation and action spaces. In the process, we carefully study the effect of different design decisions when pretraining GRPs; we evaluate how the choice of data distribution, model architecture, and policy formulation affects the quality of the pretrained GRP. Our evaluation highlights the utility of scale and flexibility: our best models are those trained on the widest data mixtures, with the least restrictive inductive biases, and with policy objectives that can fit the diversity of behaviors in the pretraining data.

Along with this paper, we release all resources required to train, use, reproduce, and finetune an Octo model. We provide pretrained Octo model checkpoints with 27M and 93M parameters that, out of the box, support multiple RGB camera inputs as well as both language and goal image task specification. We also provide scripts for finetuning these models on new domains, as well as our complete pretraining pipeline, including optimized data loaders, transformer implementations for multimodal inputs, and tools to monitor training progress.

II. Related Work

Many works train policies using a large dataset of trajectories collected from a robot, from early efforts using autonomous data collection for scaling policy training [71, 48, 41, 19, 27, 30] to more recent efforts that explore the combination of modern transformer-based policies with large demonstration datasets [10, 40, 98, 28, 83, 86]. These works primarily focus on a single embodiment, while Octo trains policies on robot datasets assembled across *multiple* embodiments, increasing the effective size of the training dataset and allowing finetuning to a range of robot setups.

More recently, papers have focused on broadening the generalization abilities of robot policies. Multiple works leverage diverse non-robot data or pretrained vision-language foundation models to boost policy generalization to new scenes and tasks [86, 103, 96, 16, 38, 11, 84, 36, 4, 37, 7, 3, 46, 15, 23]. More closely related to Octo are recent works that train robot policies across data from multiple robot embodiments: the GNM model [81, 80] generalizes across robot navigation setups while RoboCat [9] and RT-X [67] control multiple single-arm manipulation robots. While these models deliver impressive policy learning results, a key issue is their lack of flexibility: they typically require users to stick to the sensory inputs and action space used during pretraining and do not support adaptation to new observation and action spaces. Furthermore, the largest models are not publicly accessible. Octo differs from these works in multiple aspects: it is trained on a larger and more diverse robot data mix, it supports a wider range of downstream applications via efficient finetuning to new robot setups, and it is fully open source and reproducible.

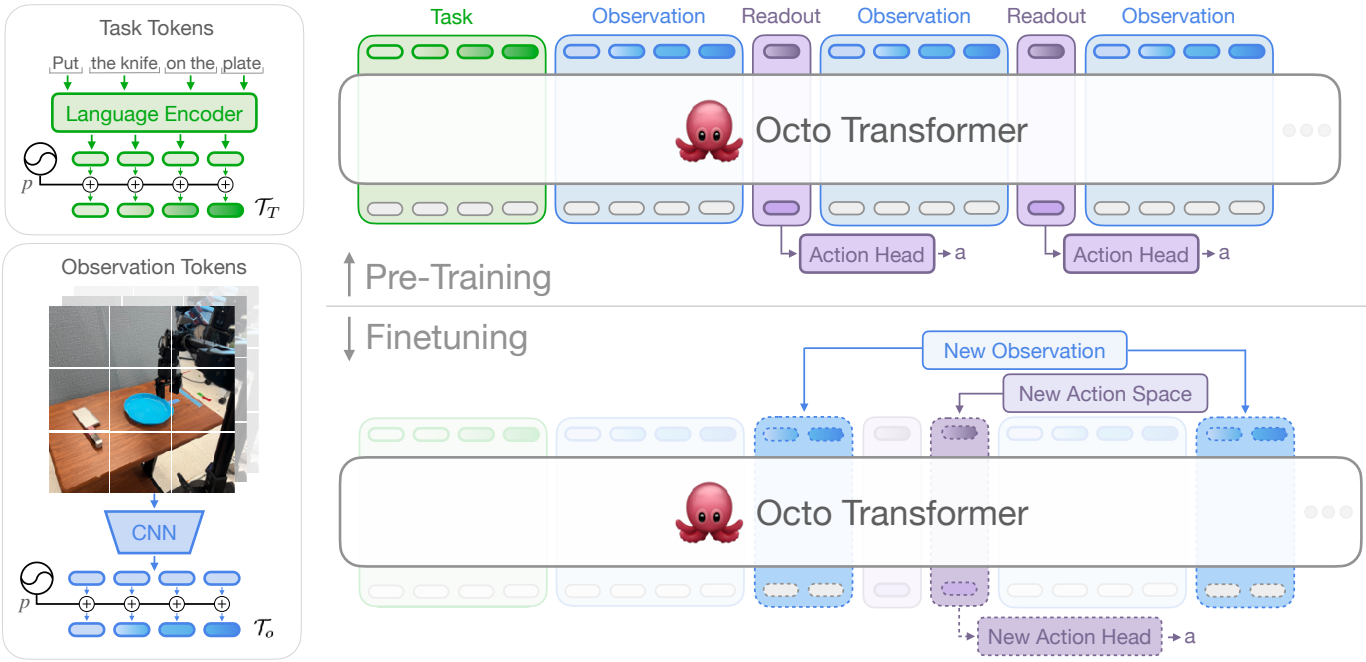

Fig. 2: **Model architecture. Left**: Octo tokenizes task descriptions (green) and input observations (blue) using a pretrained language model and a lightweight CNN, respectively. **Top**: The transformer backbone processes the sequence of task and observation tokens and produces readout tokens (purple) that get passed to output heads to produce actions. **Bottom**: The block-wise attention structure of the transformer allows us to add or remove inputs and outputs during finetuning: for example, we can add new observations (blue, dashed) or action spaces (purple, dashed) without modifying any pretrained parameters.

Octo's design is inspired by several recent advances in robot imitation learning and scalable transformer training, including the use of denoising diffusion objectives [34] for action decoding [17, 31, 85], the prediction of "action chunks", i.e., sequences of future actions [98, 17, 28], and model layouts and learning rate schedules inspired by the literature on scalable vision transformer training [22, 97]. Our work is the first to leverage these approaches in the context of learning cross-embodied generalist policies and we find that they can lead to substantial performance improvements. In our evaluation, we present ablations to assess the importance of these components, alongside a more comprehensive list of what we found to be (un)important in Appendix E; we hope our findings are useful for future research on generalist policy learning.

A key ingredient for training generalist robot policies is robot training data. In contrast to vision and language data that can be scraped from the web, obtaining robot data at scale is challenging and often involves significant investments in hardware and human labor. There are multiple large robot navigation and autonomous driving datasets [29, 95, 13, 87, 80, 43, 89]. In recent years, there have also been multiple efforts for building robot *manipulation* datasets of increasing scale and diversity, either collected via scripted and autonomous policies [19, 41, 42, 12, 71, 30] or human teleoperation [59, 60, 25, 90, 39, 10, 26, 6, 77, 63, 79]. Octo is trained on the Open X-Embodiment dataset [67], a recent effort that pooled many of these aforementioned robot datasets. The Open-X dataset contains approximately 1.5M robot episodes, of which we curate 800k for Octo training. We note that the RT-X model [67] used a more restricted subset of 350K episodes, so to the best of our knowledge, Octo is trained on the largest robotics manipulation demonstration dataset to date.

III. The Octo Model

In this section, we describe the Octo model, our open-source generalist robot policy that can be adapted to new robots and tasks — including new sensory inputs and action spaces — via finetuning. We discuss the key design decisions, training objectives, training dataset, and infrastructure. The design of the Octo model emphasizes flexibility and scale: it supports a variety of commonly used robots, sensor configurations, and actions while providing a generic and scalable recipe that can be trained on large amounts of data. It also supports natural language instructions, goal images, observation histories, and multi-modal, chunked action prediction via diffusion decoding [17]. Furthermore, we designed Octo specifically to enable efficient finetuning to new robot setups, including robots with different action spaces and different combinations of cameras and proprioceptive information. This design was selected to make Octo a flexible and broadly applicable generalist robot policy that can be utilized for a variety of downstream robotics applications and research projects.

A. Architecture

At its core, Octo is a transformer-based policy π. It consists of three key parts: **input tokenizers** that transform

language instructions ℓ, goals g, and observation sequences $o_1, \ldots, o_H$ into tokens $\left[\mathcal{T}_l, \mathcal{T}_g, \mathcal{T}_o\right]$ (Fig. 2, left); a **transformer backbone** that processes the tokens and produces embeddings $e_l, e_g, e_o = T(\mathcal{T}_l, \mathcal{T}_g, \mathcal{T}_o)$ (Fig. 2, top); and **readout heads** $R(e)$ that produce the desired outputs, i.e., actions a.

Task and observation tokenizers: We convert task definitions (e.g., language instructions ℓ and goal images g) and observations o (e.g., wrist and third-person camera streams) into a common "tokenized" format using modality-specific tokenizers (see Fig. 2, left):

- **Language inputs** are tokenized, then passed through a pretrained transformer that produces a sequence of language embedding tokens. We use the `t5-base` (111M) model [74].
- **Image observations and goals** are passed through a shallow convolution stack, then split into a sequence of flattened patches [22].

We assemble the input sequence of the transformer by adding learnable position embeddings p to task and observation tokens and then arranging them sequentially $\left[\mathcal{T}_T, \mathcal{T}_{o,1}, \mathcal{T}_{o,2}, \ldots\right]$.

Transformer backbone and readout heads: Once the inputs have been cast to a unified token sequence, they are processed by a transformer (see Fig. 2, top). This is similar to prior works that train transformer-based policies on sequences of observations and actions [92, 73]. The attention pattern of the Octo transformer is block-wise masked: observation tokens can only attend causally to tokens from the same or earlier time steps $\mathcal{T}_{o,0:t}$ as well as task tokens $\mathcal{T}_T$ (green). Tokens corresponding to non-existing observations are fully masked out (e.g., a dataset without language instructions). This modular design enables us to add and remove observations or tasks during finetuning (see below). In addition to these input token blocks, we insert learned *readout tokens* $\mathcal{T}_{R,t}$ (purple). A readout token at $\mathcal{T}_{R,t}$ attends to observation and task tokens before it in the sequence, but is not attended to by *any* observation or task token — hence, they can only passively read and process internal embeddings without influencing them. Readout tokens act similarly to the `[CLS]` token in BERT, serving as a compact vector embedding of the observation sequence thus far. A lightweight "action head" that implements the diffusion process is applied to the embeddings for the readout tokens. This action head predicts a "chunk" of several consecutive actions, similar to prior work [98, 17].

Our design allows us to flexibly add new task and observation inputs or action output heads to the model during downstream finetuning. When adding new tasks, observations, or loss functions downstream, we can wholly retain the pretrained weights for the transformer, only adding new positional embeddings, a new lightweight encoder, or the parameters of the new head as necessitated by the change in specification (see Fig. 2, bottom). This is in contrast to prior architectures [10, 81], where adding or removing an image input or changing the task specification would require re-initializing or re-training large components of the pre-trained model.

This flexibility is crucial to make Octo a truly "generalist" model: since we cannot cover all possible robot sensor and action configurations during pretraining, being able to adapt Octo's inputs and outputs during finetuning makes it a versatile tool for the robotics community. Prior model designs that use standard transformer backbones or fuse visual encoders with MLP output heads lock in the type and order of inputs expected by the model. In contrast, switching the observation or task for Octo *does not* require re-initializing most of the model.

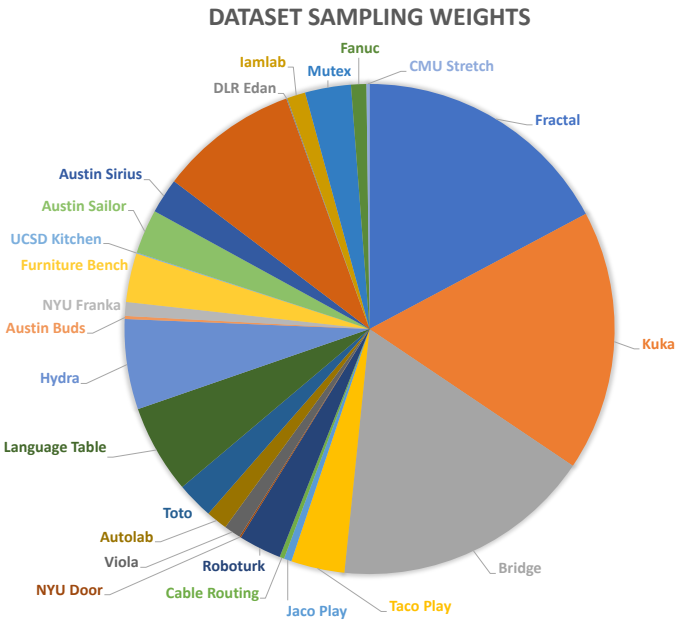

Fig. 3: **Training dataset composition.** We curate a subset of 25 datasets from the Open X-Embodiment dataset that have image observations, end-effector actions, and show diverse behaviors. The pie chart visualizes the fractions that each dataset contributes to every training batch on average. The dataset weights are determined by the number of samples in each dataset with small modifications to balance dataset size and diversity (see Section III-B for details).

B. Training data

We train Octo on a mixture of 25 datasets from the Open X-Embodiment Dataset [67], a diverse collection of robot learning datasets. Our training mixture includes demonstration data of a variety of tasks from several robot embodiments and scenes. These datasets are heterogeneous not just in terms of the robot type, but also in the sensors (e.g., including or not including wrist cameras) and labels (e.g., including or not including language instructions). See Fig. 3 and Appendix C for the detailed mixture. To create our training mixture D, we curate the data by first removing all Open-X datasets that contain no image streams, as well as those that do not use delta end-effector control. We also remove datasets that are too repetitive, have a low image resolution, or consist of excessively niche tasks. For the remaining datasets, we roughly categorize them into "more diverse" and "less diverse" datasets based on the tasks and environments, and then double the weight of the more diverse datasets during training. We also down-weight a few datasets with many repetitive episodes to avoid dominating

the mixture. Finally, we zero-pad any missing camera channels and align the gripper action spaces between the datasets such that a gripper command of +1 means "the gripper is open" and 0 means "the gripper is closed." While we found the resulting training mixture to work well, future work should perform a more thorough analysis of data mixture quality for pretraining general robot policies.

C. Training objective

We use a conditional diffusion decoding head to predict continuous, multi-modal action distributions [34, 17]. Importantly, only one forward pass of the transformer backbone is performed per action prediction, after which the multi-step denoising process is carried out entirely within the small diffusion head. We found this policy parameterization to outperform policies trained with MSE action heads or discretized action distributions [10] in both zero-shot and finetuning evaluations. To generate an action, we sample a Gaussian noise vector $x^K \sim \mathcal{N}(0, I)$ and apply K steps of denoising with a learned denoising network $\epsilon_\theta(x^k, e, k)$ that is conditioned on the output x^k of the previous denoising step, the step index k, and the output embedding e of the transformer action readout:

$$x^{k-1} = \alpha(x^k - \gamma\epsilon_\theta(x^k, e, k) + \mathcal{N}(0, \sigma^2 I)). \quad (1)$$

The hyperparameters α, γ, and σ correspond to the noise schedule: we use the standard cosine schedule from [66]. We train the diffusion head using the standard DDPM objective first proposed in [34], where we add Gaussian noise to the dataset actions and train the denoising network $\epsilon_\theta(x^k, e, k)$ to reconstruct the original action. For a detailed explanation of diffusion policy training, see Chi et al. [17]. We list all hyperparameters in Appendix D.

We use the same diffusion training objective during finetuning and update the full model, a recipe which outperformed those that freeze subsets of the pretrained parameters. In all finetuning experiments, we employ the same recipe: given a small target domain dataset with around 100 trajectories, we finetune for 50k steps using a cosine decay learning rate decay with linear warmup.

D. Training Details

We trained two variants of our model: Octo-Small with a transformer backbone that mirrors the size of a ViT-S, and Octo-Base with a transformer backbone that mirrors the size of a ViT-B [22].

We use the AdamW optimizer [51] with an inverse square root decay learning rate schedule [97], with weight decay of 0.1 and gradient clipping of 1.0. The ViT-B was trained for 300k steps with a batch size of 2048 using a TPU v4-128 pod, which took 14 hours. A finetuning run of the same model on a single NVIDIA A5000 GPU with 24GB of VRAM takes approximately 5 hours and can be sped up with multi-GPU training.

We train using 2 frames of observation history; in our preliminary experiments, we found significantly diminishing gains beyond the first additional frame. We use hindsight goal relabeling [2], which selects a state uniformly from the future in the trajectory to assign as the goal image, similar to prior work [54, 90, 81, 77, 63]. We apply common image data augmentations during training, and randomly zero out the language instruction or goal image per training example to enable Octo to be conditioned on *either* language instructions *or* goal images. For datasets without language annotations, we always use goal image conditioning. This enables our model to learn control mostly from self-supervised visual observations and reduces the burden on language annotation, similar to prior work on multi-context imitation learning [54, 62, 61, 63]. For more details on the choice of hyperparameters, see Appendix D.

E. Model Checkpoints & Code

We open-source all resources required to train, finetune and run our model (see https://octo-models.github.io):

- **Pretrained Octo checkpoints** for Octo-Small (27M params) and Octo-Base (93M params).
- **Finetuning scripts** for Octo models, in JAX.
- **Model pretraining pipeline** for Octo pretraining on the Open X-Embodiment dataset, in JAX.
- **Standalone data loaders** for Open X-Embodiment data, compatible with JAX and PyTorch.

We provide a simple example for loading and running a pretrained Octo model in Appendix B.

IV. Experiments

Our experiments provide an empirical analysis of Octo, evaluating its ability to serve as a general robotic foundation model across several axes:

1) Can Octo control multiple robot embodiments and solve language and goal tasks out of the box?
2) Do Octo weights serve as a good initialization for data-efficient finetuning to new tasks and robots, and does it improve over training from scratch and commonly used pretrained representations?
3) Which design decisions in Octo matter most for building generalist robot policies?

Evaluation setups: We evaluate Octo's capabilities across a representative spectrum of 9 robot learning setups at 4 institutions (see Fig. 4). We test Octo's ability to control different robots out-of-the-box ("zero-shot") for language and goal image tasks using robot setups that match the pretraining data, where all robots are controlled with delta end-effector control actions and the observation spaces are RGB images. We also evaluate Octo for data-efficient finetuning to new environments and tasks, including with new observations (force-torque inputs in "Berkeley Insertion"), new action spaces (joint position control in "Berkeley Pick-Up") and new robot embodiments ("Berkeley Coke" and "Berkeley Bimanual"). Each of the finetuning setups uses ~ 100 in-domain demonstrations and finetunes in < 5 hours on a NVIDIA A5000 GPU, using the same hyperparameters across all setups (see Appendix D). Our evaluation tasks test Octo's ability to interact with diverse objects (e.g., "WidowX BridgeV2"), solve long-horizon tasks

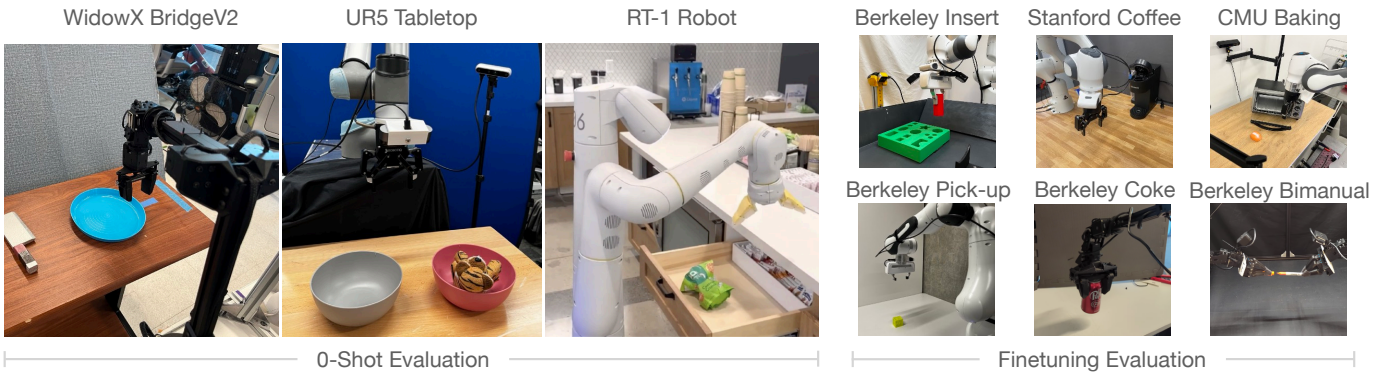

Fig. 4: **Evaluation Tasks.** We evaluate Octo on 9 real robot setups across 4 institutions. Our evaluations capture diverse object interactions (e.g., "WidowX BridgeV2"), long task horizons (e.g., "Stanford Coffee") and precise manipulation (e.g., "Berkeley Peg Insertion"). We evaluate Octo's capabilities to control robots in environments from the pretraining data out-of-the-box and to efficiently finetune to new tasks and environments with small target domain datasets. We also test finetuning with new observations (force-torque inputs for "Berkeley Peg Insertion"), action spaces (joint position control in "Berkeley Pick-Up" and "Berkeley Bimanual") and new robot embodiments (e.g., "Berkeley Bimanual" and "Berkeley Coke").

(e.g., "Stanford Coffee") and perform precise manipulation (e.g., "Berkeley Insertion"). For more details on each evaluation setup, see Appendix F.

Comparisons: We compare Octo's ability to control multiple robots out-of-the-box to the best openly available generalist robot policy, **RT-1-X** [67], using the released checkpoint. Similar to Octo, RT-1-X is pretrained on the Open X-Embodiment robot dataset and aims to control multiple robots zero-shot, thus providing a natural point of comparison. We also compare the zero-shot capabilities of Octo to **RT-2-X**, a 55 billion parameter vision-language model finetuned on the Open X-Embodiment dataset to produce robot actions. The RT-1-X and RT-2-X models [67] are trained on a more restricted subset of 350K episodes (compared to 800k episodes for Octo). We further compare Octo's performance as a policy initialization for data efficient finetuning to two common approaches: (1) training on the target domain demonstrations *from scratch* and (2) using pretrained visual representations. While a number of prior works have proposed other pretraining schemes for imitation finetuning [25, 24, 26], to our knowledge no prior method provides a pretrained *policy* that has been demonstrated to finetune successfully to *new* observation and action spaces. However, pretrained visual representations such as VC-1 [56] have been used in this way, and therefore we use these methods as another point of comparison.

For finetuning, we found that training our large transformer architecture from scratch overfit quickly on the small datasets. Instead, we obtained better from-scratch results using a canonical policy architecture employed by many prior works: a ResNet visual encoder with FiLM [70] language conditioning, combined with a small transformer action decoder trained with a diffusion objective, similar to [10, 98, 17, 55]. Our instantiation of this architecture has 28M parameters (similar to RT-1 [10]). We adopt this as our from-scratch baseline (**"ResNet+Transformer Scratch"**). We also compare to a pretrained visual representation following the procedure of Majumdar et al. [57]. A ViT-B visual encoder is initialized to the VC-1 weights [57], a state-of-the-art visual representation pretrained on 4,000 hours of ego-centric videos and ImageNet, and combined with an MLP action decoder. The full model is trained to predict expert actions using an MSE loss (**"VC-1"**).

A. *Octo Controls Multiple Robots Out-of-the-Box*

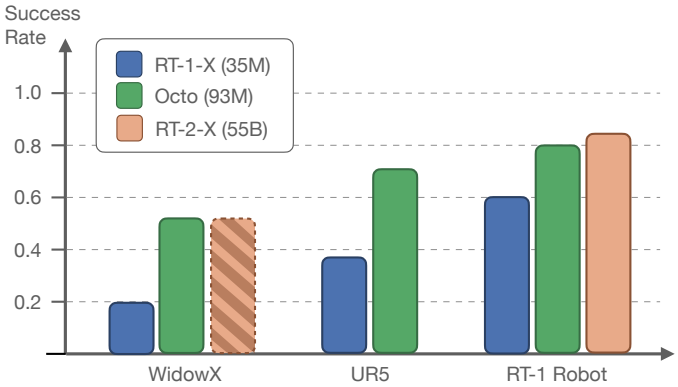

Fig. 5: **Zero-Shot Evaluation.** Out-of-the-box, Octo can control multiple robots in environments from the pretraining data. When using natural language to specify tasks, Octo outperforms RT-1-X [67], the current best openly available generalist robot policy across three different robot embodiments and setups. Octo also performs similarly to RT-2-X [103] on the tested WidowX and RT-1 Robot tasks.[1]

We compare the zero-shot manipulation capabilities of Octo, RT-1-X, and RT-2-X in Fig. 5. We evaluated on several tasks selected from the pre-training dataset including picking and

[1]For the WidowX, since RT-2-X is not openly available, we report the RT-2-X numbers from [7] (dashed bar) and use the same tasks for the Octo and RT-1-X evaluations. For the RT-1 Robot, the authors of RT-2-X kindly performed the evaluations for us.

	Berkeley Insertion*	Stanford Coffee	CMU Baking	Berkeley Pick-Up†	Berkeley Coke	Berkeley Bimanual†	Average
ResNet+Transformer Scratch	10%	45%	25%	0%	20%	20%	20%
VC-1 [57]	5%	0%	30%	0%	10%	50%	15%
Octo (Ours)	**70%**	**75%**	**50%**	**60%**	**100%**	**80%**	**72%**

TABLE I: **Finetuning Evaluation.** Octo enables data-efficient finetuning to new domains and out-performs training from scratch as well as state-of-the-art pretrained visual representations. Each domain uses ~ 100 target demonstrations and the same finetuning hyperparameters. In each domain, success rates are averaged over 20 trials. $*$: New observation input (force-torque proprioception). †: New action space (joint position control).

placing, wiping a table with a cloth, and opening and closing drawers. For each robot, we selected two language tasks from the corresponding OXE dataset and performed 10 trials per task with varying initial conditions (details in Appendix F). The chosen tasks are "in-distribution" from the pre-training data, but the evaluation requires methods to generalize to new object positions, lighting conditions, backgrounds, and distractor objects. While all methods acted reasonably across tasks in the pretraining environments, we found that on average Octo had a 29% higher success rate than RT-1-X (35M parameters). For the WidowX and RT-1 Robot evaluations, we also compared to RT-2-X (55 billion parameters) [103] and found that Octo performed similarly.

Additionally, while RT-1-X and RT-2-X only support conditioning on language instructions, Octo also supports conditioning on goal images. We evaluated our model on the WidowX tasks using goal image conditioning and found that it achieved a 25% higher success rate than when evaluated with language conditioning. This is likely because goal images provide more information about how to achieve the task. In the BridgeV2 domain, we performed a fine-grained analysis of the zero-shot capabilities in Table VII; measuring performance on setups seen in the dataset, and for novel environments, scenes, and skills. While the Octo model achieves high success on novel objects, zero-shot performance slightly degrades in a new scene, and high degradation for novel behaviors like flipping or precise insertion.

B. Octo Enables Data-Efficient Learning in New Domains

We report data-efficient finetuning results to new domains in Table I. We find that finetuning Octo leads to better policies than starting from scratch or with the pretrained VC-1 weights. On average across the six evaluation setups (detailed in Appendix F), Octo outperforms the next best baseline by 52%. Importantly, we use the same recipe and hyperparameters for fine-tuning Octo on all evaluation tasks (see Section III-C), making this a good default configuration.

The results also underline Octo's ability to accommodate new observations (force-torque inputs for "Berkeley Insertion"), action spaces (joint position control for "Berkeley Pick-Up") and new robot embodiments ("Berkeley Coke" and "Berkeley Bimanual"). This makes Octo applicable to a wide range of single and dual arm robotic manipulation problems that go beyond a single camera input and end-effector position control.

		Aggregate Performance
	Octo-Small (Ours)	**83%**
DATA	RT-X dataset mix [67]	60%
	Single robot dataset (Bridge Data)	43%
POLICY	Discretized Action Prediction [67]	18%
	Continuous Action Prediction (MSE)	35%
ARCH	Resnet-50 + Transformer[67]	70%

TABLE II: **Model Ablations.** We achieve best performance when using the ViT architecture, diffusion action head, and wide training data mixture. All evaluations are performed on the WidowX setup. Success rates are averaged over 40 trials across two language-conditioned tasks and two goal-conditioned tasks.

C. Design Decisions for Generalist Robot Policy Training

We have demonstrated the effectiveness of Octo as a zero-shot multi-robot controller and as an initialization for policy finetuning. We next analyze the effects of different design decisions on the performance of the Octo policy. Concretely, we focus on the following aspects: (1) model architecture, (2) training data, (3) training objective, and (4) model scale. Unless noted otherwise, we perform all ablations on the Octo-Small model due to our compute budget.

Model architecture: Prior transformer-based policy designs typically encode input images with large ResNet-style [32] encoders and fuse the resulting image features with a comparatively small transformer [10, 67, 81, 17, 98, 61, 83]. Instead, we opt for a "transformer-first" architecture that uses very shallow CNN patch encoders and concentrates most of the parameters and FLOPS in the transformer backbone, similar to canonical vision transformer architectures [22]. In Table II we show that this scalable architecture leads to substantially improved performance when training on the full Open X-Embodiment data mix. Importantly, we found ResNet-based architectures to perform better than ViTs when training on small datasets, e.g., in our "from scratch" comparisons, underlining that large transformer policies are uniquely suited for scalable training on diverse datasets.

Training data: Octo is trained on the most diverse cross-embodied robot dataset to date, a mix of 25 datasets that we manually curated from the Open X-Embodiment dataset [67] (see Section III-B). We ablate the impact of this training mix

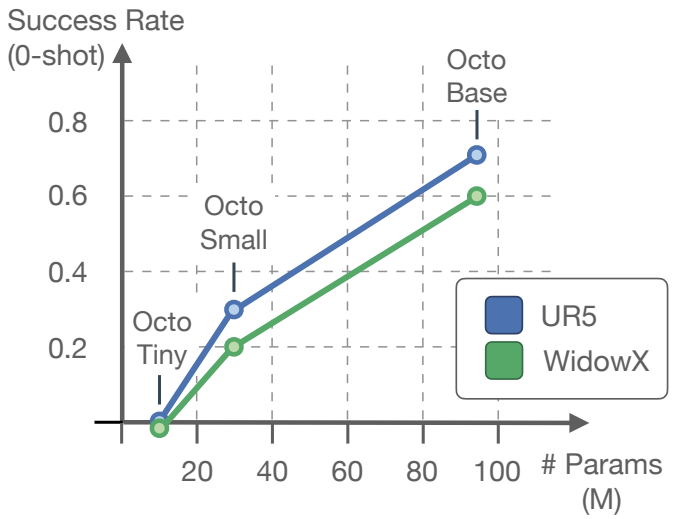

Fig. 6: **Model Scaling.** The performance of Octo improves with larger model sizes on both UR5 and WidowX tasks. Success rates are averaged over 10 trials on one language-conditioned task per robot.

by comparing to Octo models trained on a smaller mix of 11 datasets used in training the RT-X models [67] and a baseline trained only on data from the target robot domain. In Table II we show that the performance of Octo increases as we increase the number of training datasets. This suggests that expanding the data mix to even more datasets may further improve policy performance. We will leave this for future work, along with a more thorough investigation of best practices for data curation.

Training objective: We compare Octo's diffusion decoding training objective (see Section III-C) to common alternatives from prior work: simple MSE loss [8, 47] and cross-entropy loss on discretized actions [10, 103]. In Table II we find that Octo's diffusion training objective leads to substantially improved performance. This improvement is likely because the diffusion head can model multi-modal action distributions (unlike the MSE head) while retaining the precision of continuous actions (unlike the discrete head). Qualitatively, the policy acts more decisively than MSE-trained policies, and more precisely than those trained with discretized actions.

Model scale: We compare Octo models of three different sizes following the ladder of common vision transformer models [97]: Octo-Tiny (10M), Octo-Small (27M), and Octo-Base (93M). In Figure 6 we show that the zero-shot performance of the policy scales with increasing model size. We find that the Base model is more robust to initial scene configuration than the Small model, and is less prone to early grasp attempts, indicating the larger model has better visual scene perception.

V. Discussion and Future Work

We introduced Octo, a large transformer-based policy pretrained on the largest robot manipulation dataset to date, 800k robot trajectories. We demonstrated that Octo can solve a variety of tasks out-of-the-box and showed how Octo's compositional design enables finetuning to new inputs and action spaces, making Octo a versatile initialization for a wide range of robotic control problems. Apart from the model itself, we have released our full training and finetuning code, alongside tools that make it easier to train on large robot datasets.

While Octo achieves strong performance in both zero-shot and finetuning evaluations, we find that the current model still has several short-comings, which we attribute in large parts to characteristics of the training data. First, we found that the current Octo model struggles with adequately processing wrist camera information. Often finetuning results were stronger when using only a third person camera instead of combining third person and wrist camera. Additionally, we notice a large difference between language-conditioned policy performance and goal-conditioned policy performance. In both cases, a lack of the respective modalities in the training data is the likely reason: only 27% of the data contains wrist camera information and only 56% of the pretraining data contains language annotations.

Expanding the data used to train Octo is a natural avenue of improvement. Since the Open X-Embodiment dataset is comprised of optimal robot demonstrations, the current model trains via imitation; future work may consider learning from sub-optimal or online interaction data that require alternative objectives. Further, while we trained and evaluated Octo exclusively on single and dual-arm manipulators; expanding to a wider set of robots that perform navigation or mobile manipulation would be an direction of high opportunity.

While Octo represents a step towards building generalist robot policies that work out-of-the-box on diverse robot setups, there remains work to improve the model, including better language conditioning, improved support for wrist cameras, and incorporating data beyond optimal demonstrations. We hope that Octo offers a simple launchpad for researchers and practitioners to access larger robotic datasets and leverage pretrained robotics models for efficient learning of new tasks and broad generalization.

References

[1] Scale AI. Introducing scale's automotive foundation model, 2023. URL https://scale.com/blog/afm1.

[2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *NeurIPS*, 2017.

[3] Shikhar Bahl, Abhinav Gupta, and Deepak Pathak. Human-to-robot imitation in the wild. *arXiv preprint arXiv:2207.09450*, 2022.

[4] Shikhar Bahl, Russell Mendonca, Lili Chen, Unnat Jain, and Deepak Pathak. Affordances from human videos as a versatile representation for robotics. In *CVPR*, 2023.

[5] Suneel Belkhale, Yuchen Cui, and Dorsa Sadigh. Hydra: Hybrid robot actions for imitation learning. *arxiv*, 2023.

[6] Homanga Bharadhwaj, Jay Vakil, Mohit Sharma, Abhinav Gupta, Shubham Tulsiani, and Vikash Kumar.

Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. *arXiv preprint arXiv:2309.01918*, 2023.

[7] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. *arXiv preprint arXiv:2310.10639*, 2023.

[8] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[9] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, et al. Robocat: A self-improving foundation agent for robotic manipulation. *arXiv preprint arXiv:2306.11706*, 2023.

[10] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[11] Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning*, pages 287–318. PMLR, 2023.

[12] Serkan Cabi, Sergio Gómez Colmenarejo, Alexander Novikov, Ksenia Konyushkova, Scott Reed, Rae Jeong, Konrad Zolna, Yusuf Aytar, David Budden, Mel Vecerik, Oleg Sushkov, David Barker, Jonathan Scholz, Misha Denil, Nando de Freitas, and Ziyu Wang. Scaling data-driven robotics with reward sketching and batch reinforcement learning. *arXiv preprint arXiv:1909.12200*, 2019.

[13] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020.

[14] Lawrence Yunliang Chen, Simeon Adebola, and Ken Goldberg. Berkeley UR5 demonstration dataset. https://sites.google.com/view/berkeley-ur5/home.

[15] William Chen, Oier Mees, Aviral Kumar, and Sergey Levine. Vision-language models provide promptable representations for reinforcement learning. *arXiv preprint arXiv:2402.02651*, 2024.

[16] Zoey Chen, Sho Kiami, Abhishek Gupta, and Vikash Kumar. Genaug: Retargeting behaviors to unseen situations via generative augmentation. *arXiv preprint arXiv:2302.06671*, 2023.

[17] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[18] Zichen Jeff Cui, Yibin Wang, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. From play to policy: Conditional behavior generation from uncurated robot data. In *The Eleventh International Conference on Learning Representations*, 2022.

[19] Sudeep Dasari, Frederik Ebert, Stephen Tian, Suraj Nair, Bernadette Bucher, Karl Schmeckpeper, Siddharth Singh, Sergey Levine, and Chelsea Finn. Robonet: Large-scale multi-robot learning. In *Conference on Robot Learning*, pages 885–897. PMLR, 2020.

[20] Shivin Dass, Jullian Yapeter, Jesse Zhang, Jiahui Zhang, Karl Pertsch, Stefanos Nikolaidis, and Joseph J. Lim. CLVR jaco play dataset, 2023. URL https://github.com/clvrai/clvr_jaco_play_dataset.

[21] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. *NeurIPS*, 2019.

[22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[23] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.

[24] Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. *ArXiv*, abs/2304.08742, 2023. URL https://api.semanticscholar.org/CorpusID:258186973.

[25] Frederik Ebert, Yanlai Yang, Karl Schmeckpeper, Bernadette Bucher, Georgios Georgakis, Kostas Daniilidis, Chelsea Finn, and Sergey Levine. Bridge data: Boosting generalization of robotic skills with cross-domain datasets. *arXiv preprint arXiv:2109.13396*, 2021.

[26] Hao-Shu Fang, Hongjie Fang, Zhenyu Tang, Jirong Liu, Chenxi Wang, Junbo Wang, Haoyi Zhu, and Cewu Lu. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot. *Towards Generalist Robots: Learning Paradigms for Scalable Skill Acquisition@ CoRL2023*, 3:5, 2023.

[27] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2786–2793. IEEE, 2017.

[28] Zipeng Fu, Tony Z Zhao, and Chelsea Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.

[29] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.

[30] Abhinav Gupta, Adithyavairavan Murali, Dhiraj Prakashchand Gandhi, and Lerrel Pinto. Robot learning in homes: Improving generalization and reducing dataset bias. *Advances in neural information processing systems*, 31, 2018.

[31] Huy Ha, Pete Florence, and Shuran Song. Scaling up and distilling down: Language-guided robot skill acquisition. In *Conference on Robot Learning*, pages 3766–3777. PMLR, 2023.

[32] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[33] Minho Heo, Youngwoon Lee, Doohyun Lee, and Joseph J. Lim. Furniturebench: Reproducible real-world benchmark for long-horizon complex manipulation. In *Robotics: Science and Systems*, 2023.

[34] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[35] Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. Gaia-1: A generative world model for autonomous driving, 2023.

[36] Chenguang Huang, Oier Mees, Andy Zeng, and Wolfram Burgard. Visual language maps for robot navigation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10608–10615. IEEE, 2023.

[37] Chenguang Huang, Oier Mees, Andy Zeng, and Wolfram Burgard. Audio visual language maps for robot navigation. In *Proceedings of the International Symposium on Experimental Robotics (ISER)*, Chiang Mai, Thailand, 2023.

[38] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.

[39] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. Bc-z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning*, pages 991–1002. PMLR, 2022.

[40] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. VIMA: Robot manipulation with multimodal prompts. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 14975–15022. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/jiang23b.html.

[41] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293*, 2018.

[42] Dmitry Kalashnikov, Jake Varley, Yevgen Chebotar, Benjamin Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman. Scaling up multi-task robotic reinforcement learning. In *5th Annual Conference on Robot Learning*, 2021.

[43] Haresh Karnan, Anirudh Nair, Xuesu Xiao, Garrett Warnell, Sören Pirk, Alexander Toshev, Justin Hart, Joydeep Biswas, and Peter Stone. Socially compliant navigation dataset (scand): A large-scale dataset of demonstrations for social navigation. *IEEE Robotics and Automation Letters*, 7(4):11807–11814, 2022.

[44] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, et al. Segment Anything, April 2023.

[45] Vikash Kumar, Rutav Shah, Gaoyue Zhou, Vincent Moens, Vittorio Caggiano, Abhishek Gupta, and Aravind Rajeswaran. Robohive: A unified framework for robot learning. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL https://openreview.net/forum?id=0H5fRQcpQ7.

[46] Teyun Kwon, Norman Di Palo, and Edward Johns. Language models as zero-shot trajectory generators. *arXiv preprint arXiv:2310.11604*, 2023.

[47] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.

[48] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International journal of robotics research*, 37(4-5):421–436, 2018.

[49] Yixin Lin, Austin S. Wang, Giovanni Sutanto, Akshara Rai, and Franziska Meier. Polymetis. https://facebookresearch.github.io/fairo/polymetis/, 2021.

[50] Huihan Liu, Soroush Nasiriany, Lance Zhang, Zhiyao Bao, and Yuke Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. In *Robotics: Science and Systems (RSS)*, 2023.

[51] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2018.

[52] Jianlan Luo, Charles Xu, Xinyang Geng, Gilbert Feng, Kuan Fang, Liam Tan, Stefan Schaal, and Sergey Levine. Multi-stage cable routing through hierarchical imitation learning. *arXiv preprint arXiv:2307.08927*, 2023.

[53] Jianlan Luo, Charles Xu, Fangchen Liu, Liam Tan, Zipeng Lin, Jeffrey Wu, Pieter Abbeel, and Sergey Levine. Fmb: a functional manipulation benchmark for generalizable robotic learning. *arXiv preprint arXiv:2401.08553*, 2024.

[54] Corey Lynch and Pierre Sermanet. Language conditioned imitation learning over unstructured data. In *RSS*, 2021.

[55] Corey Lynch, Ayzaan Wahid, Jonathan Tompson, Tianli Ding, James Betker, Robert Baruch, Travis Armstrong, and Pete Florence. Interactive language: Talking to robots in real time. *IEEE Robotics and Automation Letters*, 2023.

[56] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Yecheng Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Pieter Abbeel, Jitendra Malik, Dhruv Batra, Yixin Lin, Oleksandr Maksymets, Aravind Rajeswaran, and Franziska Meier. Where are we in the search for an artificial visual cortex for embodied intelligence? 2023.

[57] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Yecheng Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Pieter Abbeel, Jitendra Malik, et al. Where are we in the search for an artificial visual cortex for embodied intelligence? *arXiv preprint arXiv:2303.18240*, 2023.

[58] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Booher, Max Spero, Albert Tung, Julian Gao, John Emmons, Anchit Gupta, Emre Orbay, Silvio Savarese, and Li Fei-Fei. RoboTurk: A crowdsourcing platform for robotic skill learning through imitation. *CoRR*, abs/1811.02790, 2018. URL http://arxiv.org/abs/1811.02790.

[59] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Booher, Max Spero, Albert Tung, Julian Gao, John Emmons, Anchit Gupta, Emre Orbay, et al. Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conference on Robot Learning*, pages 879–893. PMLR, 2018.

[60] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.

[61] Oier Mees, Lukas Hermann, and Wolfram Burgard. What matters in language conditioned robotic imitation learning over unstructured data. *IEEE Robotics and Automation Letters*, 7(4):11205–11212, 2022.

[62] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters (RA-L)*, 7(3):7327–7334, 2022.

[63] Oier Mees, Jessica Borja-Diaz, and Wolfram Burgard. Grounding language with visual affordances over unstructured data. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, London, UK, 2023.

[64] Russell Mendonca, Shikhar Bahl, and Deepak Pathak. Structured world models from human videos. *CoRL*, 2023.

[65] Soroush Nasiriany, Tian Gao, Ajay Mandlekar, and Yuke Zhu. Learning and retrieval from prior data for skill-based imitation learning. In *Conference on Robot Learning (CoRL)*, 2022.

[66] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021.

[67] Open X-Embodiment Collaboration, Abhishek Padalkar, Acorn Pooley, Ajinkya Jain, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anikait Singh, Anthony Brohan, Antonin Raffin, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Brian Ichter, Cewu Lu, Charles Xu, Chelsea Finn, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Chuer Pan, Chuyuan Fu, Coline Devin, Danny Driess, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Federico Ceola, Fei Xia, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Giulio Schiavi, Hao Su, Hao-Shu Fang, Haochen Shi, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jaehyung Kim, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jiajun Wu, Jialin Wu, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jitendra Malik, Jonathan Tompson, Jonathan Yang, Joseph J. Lim, João Silvério, Junhyek Han, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Zhang, Keyvan Majd, Krishan Rana, Krishnan Srinivasan, Lawrence Yunliang Chen, Lerrel Pinto, Liam Tan, Lionel Ott, Lisa Lee, Masayoshi Tomizuka, Maximilian Du, Michael Ahn, Mingtong Zhang, Mingyu Ding, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Pannag R Sanketi, Paul Wohlhart, Peng Xu, Pierre Sermanet, Priya Sundaresan, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Sherry Moore, Shikhar Bahl, Shivin Dass, Shuran Song, Sichun Xu, Siddhant Haldar, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Sudeep Dasari, Suneel Belkhale, Takayuki Osa, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard,

Xi Chen, Xiaolong Wang, Xinghao Zhu, Xuanlin Li, Yao Lu, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yueh hua Wu, Yujin Tang, Yuke Zhu, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zhuo Xu, and Zichen Jeff Cui. Open X-Embodiment: Robotic learning datasets and RT-X models. https://arxiv.org/abs/2310.08864, 2023.

[68] OpenAI. GPT-4 Technical Report, March 2023.

[69] Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation, 2021.

[70] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[71] Lerrel Pinto and Abhinav Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 3406–3413. IEEE, 2016.

[72] Gabriel Quere, Annette Hagengruber, Maged Iskandar, Samuel Bustamante, Daniel Leidner, Freek Stulp, and Joern Vogel. Shared Control Templates for Assistive Robotics. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, page 7, Paris, France, 2020.

[73] Ilija Radosavovic, Baifeng Shi, Letian Fu, Ken Goldberg, Trevor Darrell, and Jitendra Malik. Robot learning with sensorimotor pre-training. *Conference on Robot Learning*, 2023.

[74] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

[75] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barth-maron, Mai Giménez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. *Transactions on Machine Learning Research*, 2022.

[76] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, April 2022.

[77] Erick Rosete-Beas, Oier Mees, Gabriel Kalweit, Joschka Boedecker, and Wolfram Burgard. Latent plans for task agnostic offline reinforcement learning. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.

[78] Saumya Saxena, Mohit Sharma, and Oliver Kroemer. Multi-resolution sensing for real-time control with vision-language models. In *7th Annual Conference on Robot Learning*, 2023. URL https://openreview.net/forum?id=WuBv9-IGDUA.

[79] Nur Muhammad Mahi Shafiullah, Anant Rai, Haritheja Etukuru, Yiqian Liu, Ishan Misra, Soumith Chintala, and Lerrel Pinto. On bringing robots home, 2023.

[80] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, Noriaki Hirose, and Sergey Levine. Gnm: A general navigation model to drive any robot. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7226–7233. IEEE, 2023.

[81] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. ViNT: A foundation model for visual navigation. In *7th Annual Conference on Robot Learning*, 2023. URL https://arxiv.org/abs/2306.14846.

[82] Rutav Shah, Roberto Martín-Martín, and Yuke Zhu. MUTEX: Learning unified policies from multimodal task specifications. In *7th Annual Conference on Robot Learning*, 2023. URL https://openreview.net/forum?id=PwqiqaaEzJ.

[83] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799. PMLR, 2023.

[84] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023.

[85] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration. *arXiv preprint arXiv:2310.07896*, 2023.

[86] Austin Stone, Ted Xiao, Yao Lu, Keerthana Gopalakrishnan, Kuang-Huei Lee, Quan Vuong, Paul Wohlhart, Sean Kirmani, Brianna Zitkovich, Fei Xia, et al. Open-world object manipulation using pre-trained vision-language models. In *7th Annual Conference on Robot Learning*, 2023.

[87] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2446–2454, 2020.

[88] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and Efficient Foundation Language Models, February 2023.

[89] Samuel Triest, Matthew Sivaprakasam, Sean J Wang, Wenshan Wang, Aaron M Johnson, and Sebastian Scherer. Tartandrive: A large-scale dataset for learning off-road dynamics models. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 2546–2552. IEEE, 2022.

[90] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale, 2023.

[91] Wayve. Lingo: Natural language for autonomous driving, 2023. URL https://wayve.ai/thinking/lingo-natural-language-autonomous-driving/.

[92] Philipp Wu, Arjun Majumdar, Kevin Stone, Yixin Lin, Igor Mordatch, Pieter Abbeel, and Aravind Rajeswaran. Masked trajectory models for prediction, representation, and control. *International Conference on Machine Learning*, 2023.

[93] Ge Yan, Kris Wu, and Xiaolong Wang. ucsd kitchens Dataset. August 2023.

[94] Jonathan Heewon Yang, Dorsa Sadigh, and Chelsea Finn. Polybot: Training one policy across robots while embracing variability. In *7th Annual Conference on Robot Learning*, 2023. URL https://openreview.net/forum?id=HEIRj51lcS.

[95] Fisher Yu, Haofeng Chen, Xin Wang, Wenqi Xian, Yingying Chen, Fangchen Liu, Vashisht Madhavan, and Trevor Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2636–2645, 2020.

[96] Tianhe Yu, Ted Xiao, Austin Stone, Jonathan Tompson, Anthony Brohan, Su Wang, Jaspiar Singh, Clayton Tan, Jodilyn Peralta, Brian Ichter, et al. Scaling robot learning with semantically imagined experience. *arXiv preprint arXiv:2302.11550*, 2023.

[97] Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12104–12113, 2022.

[98] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[99] Gaoyue Zhou, Victoria Dean, Mohan Kumar Srirama, Aravind Rajeswaran, Jyothish Pari, Kyle Hatch, Aryan Jain, Tianhe Yu, Pieter Abbeel, Lerrel Pinto, Chelsea Finn, and Abhinav Gupta. Train offline, test online: A real robot learning benchmark, 2023.

[100] Xinghao Zhu, Ran Tian, Chenfeng Xu, Mingyu Ding, Wei Zhan, and Masayoshi Tomizuka. Fanuc manipulation: A dataset for learning-based manipulation with fanuc mate 200id robot. 2023.

[101] Yifeng Zhu, Peter Stone, and Yuke Zhu. Bottom-up skill discovery from unsegmented demonstrations for long-horizon robot manipulation. *IEEE Robotics and Automation Letters*, 7(2):4126–4133, 2022.

[102] Yifeng Zhu, Abhishek Joshi, Peter Stone, and Yuke Zhu. Viola: Imitation learning for vision-based manipulation with object proposal priors, 2023.

[103] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *7th Annual Conference on Robot Learning*, 2023.

Appendix A
Contributions

Dibya Ghosh: led model development, proposed and implemented large parts of the final model design, babysitted the training runs and touched all parts of the codebase, helped with model evaluations and tech report writing.

Homer Walke: led model evaluations, designed the main Bridge evaluation benchmark, contributed to the initial model implementation and ran many of the evals for this tech report.

Karl Pertsch: managed the overall project, led Open-X data integration and curation, led writing of this tech report, contributed to model development and implementation and ran model evaluations for the tech report.

Kevin Black: led data loading and training infrastructure, managed TPU pod training, created the project website, contributed to model development and implementation, and helped with robot evaluations and tech report writing.

Oier Mees: ran countless ablations for model development, contributed to model implementation, helped with evals and writing of the tech report.

Sudeep Dasari: contributed the model evaluations at CMU, experimented with pretrained encoders.

Joey Hejna: contributed the model evaluations at Stanford.

Tobias Kreiman: contributed model evaluations in simulated environments.

Ria Doshi: contributed to explorations into cross-morphology training.

Charles Xu: contributed the model evaluations for Berkeley Peg Insert.

Jianlan Luo: contributed the model evaluations for Berkeley Peg Insert.

You Liang Tan: helped diagnose and resolve bottlenecks in data loading.

Lawrence Yunliang Chen: helped set up and run UR5 finetuning experiments at Berkeley AutoLab.

Pannag Sanketi, Quan Vuong, Ted Xiao: contributed model evaluations on the Google Robot.

Dorsa Sadigh, Chelsea Finn, Sergey Levine: provided guidance throughout the project and feedback on the writing of this tech report.

Appendix B
Octo Code Example

Loading a pretrained Octo model and performing inference requires little code:

```python
import jax
from octo.model.octo_model import OctoModel

model = OctoModel.load_pretrained("hf://rail-berkeley/octo-base")
print(model.get_pretty_spec()) # Print out the input-output spec
observation = {"image_primary": img}
task = model.create_tasks(texts=["pick up the fork"])
action = model.sample_actions(
    observation, task, rng=jax.random.PRNGKey(0))
```

Listing 1: Example Python code to perform inference with a pretrained Octo model.

Appendix C
Data mixture

We list the detailed training mixture used for training the Octo models in Table III. The sampling weights are mostly determined by the relative size of the datasets with a few manual adjustments (see Section III-B).

Octo Pretraining Dataset Mixture	
Fractal [10]	17.0%
Kuka [41]	17.0%
Bridge[25, 90]	17.0%
BC-Z [39]	9.1%
Stanford Hydra Dataset [5]	6.0%
Language Table [55]	5.9%
Taco Play [77, 63]	3.6%
Furniture Bench Dataset [33]	3.3%
UTAustin Mutex [82]	3.0%
Austin Sailor Dataset [65]	2.9%
Roboturk [58]	2.8%
Toto [99]	2.4%
Austin Sirius Dataset [50]	2.3%
Berkeley Autolab UR5 [14]	1.5%
IAMLab CMU Pickup Insert [78]	1.2%
Viola [102]	1.2%
Berkeley Fanuc Manipulation [100]	1.0%
NYU Franka Play Dataset [18]	0.9%
UCSD Kitchen Dataset [93]	<0.1%
Jaco Play [20]	0.6%
Berkeley Cable Routing [52]	0.3%
Austin Buds Dataset [101]	0.3%
CMU Stretch [64]	0.2%
NYU Door Opening [69]	0.1%
DLR EDAN Shared Control [72]	0.1%

TABLE III: Octo pretraining data mixture using datasets from the Open X-Embodiment dataset [67].

Appendix D
Training Hyperparameters

We mostly follow documented practices for training vision transformers [97]. We use the AdamW optimizer [51] with an inverse square root decay learning rate schedule [97] and linear learning rate warm-up. We list hyperparamaters used during training in Table IV and the model parameters for the different sizes in Table V. We apply standard image augmentations during training. Concretely, for the 3rd person camera we apply stochastic crops followed be a resize to 256×256, followed by color jitter. Finally, we normalize the input image to have pixels with float values between -1.0 and 1.0. For the wrist camera, we apply the same procedure except without the random crop and resizing to 128×128 instead.

Hyperparameter	Value
Learning Rate	3e-4
Warmup Steps	2000
LR Scheduler	reciprocal square-root
Weight Decay	0.1
Gradient Clip Threshold	1
Batch Size	2048

TABLE IV: Hyperparameters used during training.

The images are passed through a shallow convolution stack, then split into a sequence of flattened patches [22] of size 16×16. This results in 256 tokens for the 3rd person camera images and 64 tokens for the wrist camera images. For datasets containing language annotations, we use a pretrained `t5-base` (111M) transformer model [74] that produces a sequence of 16 language embedding tokens.

Model	Layers	Hidden size D	MLP size	Heads	Params
Octo-Small	12	384	1536	6	27M
Octo-Base	12	768	3072	12	93M

TABLE V: Architecture details of Octo model variants.

The diffusion action head consists of a 3-layer MLP with a hidden dimension of 256, residual connections, and layer normalization. We use the standard DDPM objective as introduced by [34] with a cosine noise schedule [66] and 20 diffusion steps.

APPENDIX E
THINGS THAT WORKED AND DID NOT WORK (YET)

Things we found improved performance:

- **Adding history during pretraining**: Models with one frame of history as context performed better in zero-shot evals than models pretrained without history. We did not observe benefits of increasing the history length further on the few tasks we evaluated on, though other tasks may benefit.
- **Using action chunking**: We found it helpful to use "action chunking" [98], i.e., to predict multiple actions into the future, for getting more coherent policy movements. In our evaluations, we did not find temporal ensembling of future actions to provide additional benefits beyond receding horizon control.
- **Decreasing patch size** Tokenizing images into patches of size 16×16 led to improved performance over patches of size 32×32, particularly for grasping and other fine-grained tasks. This does add compute complexity (the number of tokens is $4\times$), so understanding how to balance compute costs and resolution remains a problem of interest.
- **Increasing shuffle buffer size**: Loading data from 25 datasets in parallel is a challenge. Specifically, we found that achieving good shuffling of frames during training was crucial — zero-shot performance with a small shuffle buffer (20k) and trajectory-level interleaving suffered significantly. We solved this issue by shuffling and interleaving frames from different trajectories *before* decoding the images, allowing us to fit a much larger shuffle buffer (up to 500k). We also subsample at most 100 randomly chosen steps from each training trajectory during data loading to avoid "over-crowding" the shuffle buffer with single, very long episodes.

Things that did not work (yet):

- **MSE Action Heads**: Replacing our diffusion decoding head with a simple L2 loss led to "hedging" policies that move very slowly and e.g., fail to rotate the gripper in WidowX evaluations.
- **Discrete Action Heads**: Discretizing actions into 256 bins per dimension and training with cross-entropy loss like in Brohan et al. [10] led to more "decisive" policies, yet they lacked precision and often missed the grasp.
- **ResNet Encoders**: did not scale as well to larger datasets in our evaluations (see Table II), though they did outperform our ViT architecture when training from scratch on a small dataset (around 100 demonstrations).
- **Pretrained Encoders**: ImageNet pretrained ResNet encoders did not provide benefit on zero-shot evals, though may be confounded with ResNet architectures underperforming as mentioned above.
- **Relative Gripper Action Representation**: When aligning the gripper action representations of the different datasets, we tried (A) absolute gripper actions, i.e., actions are +1 when the gripper is open and 0 if it is closed, and (B) relative gripper actions, i.e., gripper action is +1/0 only in the timestep when the gripper opens/closes and 0.5 otherwise. We found that the latter tends to open/close the gripper less often since most of the training data represents "do not change gripper" actions, leading to a slightly higher grasp success rate. At the same time, the relative representation led to less retrying behavior after a grasp failed, which was ultimately worse. Thus, we chose the absolute gripper action representation.
- **Adding Proprioceptive Inputs**: Policies trained with propioceptive observations seemed generally worse, potentially due to a strong correlation between states and future actions. We hypothesize this might be due to a causal confusion between the proprioceptive information and the target actions [21].
- **Finetuning Language Model**: In order to improve the visuo-lingual grounding of Octo we experimented with: i) varying sizes of the T5 encoder [74]: `small` (30M), `base` (111M), and `large` (386M) as well as ii) finetuning the last two layers of the encoder. Using the frozen `base` model resulted in the best language-conditioned policies. We did not find improvements when using larger encoders or finetuning the encoder. We hypothesize this might be due to the lack of rich, diverse, free-form language annotations in most of the datasets.

APPENDIX F
EXPERIMENTAL SETUPS

A. Zero-Shot Evaluations

WidowX BridgeV2: Uses the setup of Walke et al. [90], in which a Trossen WidowX 250 6-DOF robot performs diverse manipulation tasks. The observation consists of a single third person camera stream and the action space is end-effector position deltas. We evaluated two language-conditioned tasks

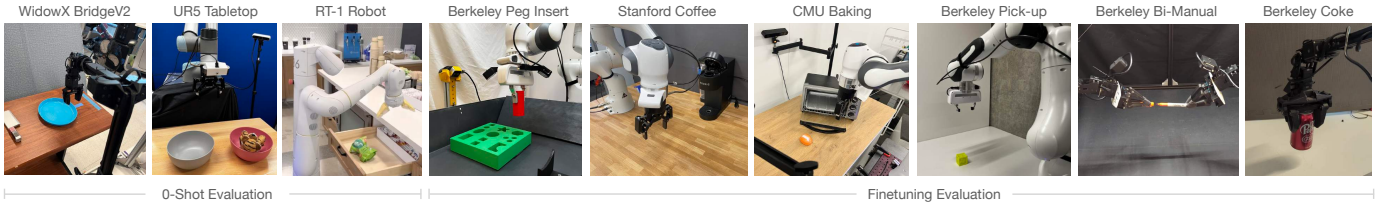

Fig. 7: **Evaluation Tasks.** Replicated from the main text for convenience. We evaluate Octo on 9 real robot setups across 4 institutions in zero-shot and finetuning scenarios.

in which a the robot needs to "place the carrot on plate", and "put the eggplant in the pot." While these tasks are in-distribution, the policy must still generalize to novel object positions. We performed 10 trials per task and varied objects positions between trials.

UR5: Uses the setup of Chen et al. [14], in which a UR5 robot arm performs multiple table top manipulation tasks. The observation consists of a single third person camera stream and the action space is end-effector position deltas. We evaluated twp language-conditioned tasks: picking a toy tiger from a bowl and placing it into a different bowl as well as wiping a table with a cloth. While these tasks are in-distribution, the policy must still generalize to novel object positions, distractor objects, and lighting. Since the training data was collected months ago and the robot setup was taken down and re-assembled, the policy must also generalize to other miscellaneous changes in the environment like a slightly different camera view and background. We performed 10 trials per task and varied objects positions between trials.

RT-1 Robot: Uses the setup of Brohan et al. [10], in which a proprietary robot performs multiple table top and furniture manipulation tasks. The observation consists of a single third person camera stream and the action space is end-effector position deltas. We evaluated on the task of picking up a 7up can, apple, blue chip bag, or brown chip bag, as well as the task of opening or closing drawers on a cabinet. While these tasks are in-distribution, the policy must still generalize to novel object positions. Additionally, since the robot setup was moved between buildings, the policy must also generalize to miscellaneous changes in the environment like a slightly different camera view and background. We performed 10 trials per task and varied objects positions between trials.

B. Model Ablations

All of our model ablations were evaluated on the WidowX setup. We present a more detailed breakdown of the success rates per task in Table VI. We evaluated on two language-conditioned tasks (put carrot on plate and put eggplant in pot) and two goal-conditioned tasks (put bread on plate and put spoon on glove). The goal-conditioned tasks contain objects not seen in the Bridge dataset. Additionally, we analyze the generalization capabilities of Octo across several different axes, such as novel objects, novel environments and novel skills in Table VII.

C. Finetuning Evaluations

CMU Baking: The robot must pick up the toy bread object, place it in the toaster, and shut the toaster. This task requires generalization across initial positions (of both the toaster and object) and the shape of the target toy bread object. We use an end-effector delta action space (Cartesian position + rotation delta). Observations come from the 3rd-person front-facing Zed camera. Actions are predicted at 15 Hz, and executed on the robot using the R2D2 Franka controller. The finetuning dataset consists of 120 demos collected via expert VR tele-operation, and every policy was evaluated using 20 trials (4 novel test objects with 5 positions each).

Stanford Coffee: The robot is tasked with picking up one of four different Keurig Coffee Pods and placing it inside of a Keurig machine. This task requires both generalization across initial positions and colors of the coffee pod, as well as precise placement in the Keurig machine. We use an end effector delta action space with an open source controller running at 10 Hz based on Polymetis [49] (found here). We use only a single 3rd-person wrist observation. Our training dataset contained 118 expert demonstrations from varied coffee pods and positions collected via VR tele-operation. We evaluated policies for 20 episodes, five episodes for each of four different color coffee pods.

Berkeley Peg Insertion: The task is to insert a pre-grasped 3D-printed peg into a matching slot on a 3D-printed board inside the bin. The matching tolerance between the peg and the hole is 1.5mm; which makes it a contact-rich precise part-mating task. The robot must learn an appropriate policy to "search" for the matching opening through contact, which necessitates the use of force/torque measurements. The observation space of the policy consists of a single side-view camera image, the end-effector twist, and the end-effector force/torque reading. The policy sends action commands as the robot's end-effector twists at 5 HZ, tracked at 1000 HZ by a low-level impedance controller. Our finetuning dataset is composed of 100 human demonstrations from the FMB dataset [53]. We evaluated trained policies for 20 trials with randomized board positions.

Berkeley Pick Up: We use the setup of Radosavovic et al. [73]. The robot needs to pick up a block from a table top surface after being trained on a dataset of 100 pickups of various objects. The robot uses joint position control with an underlying Polymetis control stack [49] (here). It is conditioned on a wrist camera input image, as well as the proprioceptive

		Language-conditioned		Goal-conditioned		
		Put carrot on plate	Put eggplant in pot	Put bread on plate	Put spoon on glove	Average
	Octo-small (Ours)	80%	90%	70%	90%	**83%**
DATA	RT-X dataset mix [67]	80%	80%	40%	40%	60%
	Single robot dataset (Bridge Data)	20%	70%	60%	20%	43%
POLICY	Discretized Action Prediction [67]	0%	20%	10%	40%	18%
	Continuous Action Prediction (MSE)	70%	30%	0%	40%	35%
ARCH	Resnet-50 + Transformer [67]	80%	60%	100%	40%	70%

TABLE VI: **Model Ablations.** We achieve best performance when using the ViT architecture, diffusion action head, and wide training data mixture. All evaluations are performed on the WidowX setup. Success rates are averaged over 40 trials across two language-conditioned tasks and two goal-conditioned tasks.

Generalization Type	Task	Success Rate	Average
In-distribution	Put carrot on plate Put eggplant in pot	80% 90%	85%
Novel objects	Put bread on plate Put spoon on glove	70% 90%	80%
Novel environment	Put mushroom in pot Put spoon on cloth	20% 60%	40%
Novel skill	Flip cup on its side Put block in slot	10% 0%	5%

TABLE VII: **Zero-shot Generalization Analysis.** Using the WidowX setup, we analyze the generalization capabilities of Octo-small along several different axes. Octo performs the best on in-distribution tasks and generalizes well to novel objects and environments. However, Octo struggles to generalize to skills not seen in the WidowX embodiment. Success rates are averaged over 20 trials across two tasks.

readings of the robot. We evaluted on the task of picking up the yellow cube and used 20 trials.

Berkeley Coke: A Trossen Robotics ViperX robot is tasked with picking up a coke can from a table. The ViperX is completely unseen in the training mixture, thus this task tests the model's capabilities to generalize to a new robot embodiment. The task also requires generalization across different positions of the coke can with the ViperX robot using end-effector delta control at 5Hz. The policy uses image observations from a third person and a wrist camera. Our training dataset contained 115 expert demonstrations collected via VR tele-operation from varied positions.

Berkeley Bimanual: The task requires an ALOHA bimanual robot [98], consisting of two ViperX robot arms, to pick up a sharpie marker with the right hand from the workbench and remove its cap with the left hand. Since Octo was only pre-trained with single-arm robot data, we re-initialize a new action head that maps to the 14-dimensional action space of ALOHA (2x 6 joint positions + 2x gripper position). A successful policy needs to use image observations from the left and right wrist cameras to predict precise manipulation behaviors. Best performance for both our methods and baselines was achieved by predicting an action chunk of 64 during training, and then during test-time executing 12 actions with receding horizon control before re-planning. We evaluated the uncapping task using 10 trials, where the pen was placed in a different location for each trial.